Applied Agent Engineering Foundations

Day 2 — Agents, MCP, and Governance

Dr. Anand Jayaraman

TekFrameworks / Soothsayer Analytics

© 2025 TekFrameworks

Day 1

How Machines Learn to Understand Language — and Why That Matters for RAG

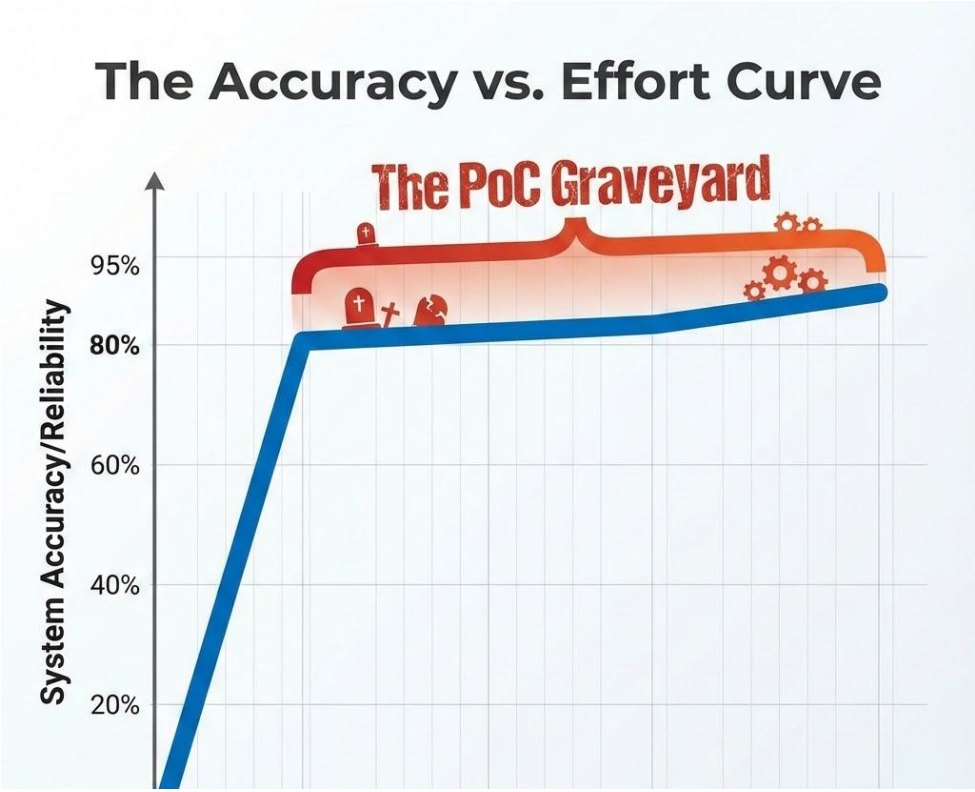
- Language is hard. Rules and keywords break on context and ambiguity.
- Word2Vec: encode meaning as numbers. Similar contexts → similar vectors. Powers Netflix, Spotify, Amazon recommendations.
- Attention solves context-sensitivity. The transformer makes it parallel. Transformers paper changed everything.
- Three training stages: unsupervised pre-training → supervised fine-tuning → RLHF alignment.
- Neural scaling laws explain the race to larger models. Capability scales predictably with size, data, and compute.
- SLMs matter for enterprise: lower latency, lower cost, easier to govern. Right-size the model to the task.

- LLM API call syntax: every call is stateless. Prompt chaining builds memory across turns.
- RAG: ground answers in your documents. Eliminates hallucination. Solves the training cutoff problem.
- Chunking strategy determines what the model sees. Too large → noise. Too small → lost context.
- Vector databases store meaning. Reranking improves the candidates that reach the context window.
- RAGAS: automated evaluation across faithfulness, context precision, and answer relevance.
- Golden sets + CI/CD pipelines catch quality drift before users do.

© 2025 TekFrameworks

The 90% Trap

- 80% of GenAI POCs do not reach production
- The last-mile problem:
  - 0% → 80% Accuracy: Easy. Takes 1 week
  - 80% → 95%: Hard. Takes 4 months
- Why they fail:
  - Latency: “takes 1min for answer”
  - Edge cases: “Works for English invoices, but breaks for French”
  - Governance: “Can’t deploy because it logs PII”
- The Reality:
  - Building the demo is 10% of the work
  - Building the system is 90% of the work



© 2025 TekFrameworks

Most Common Issues in Production RAG

Yesterday you built a working RAG pipeline. Here is what breaks when it meets real data and real users.

Stale data

Documents updated after ingestion return wrong answers. The index does not know the document changed.

Cost and latency

Repetitive queries hit the LLM every time. 30–40% of enterprise queries are identical.

Retrieval failures

Exact attribute lookups and multi-hop questions defeat vector search.

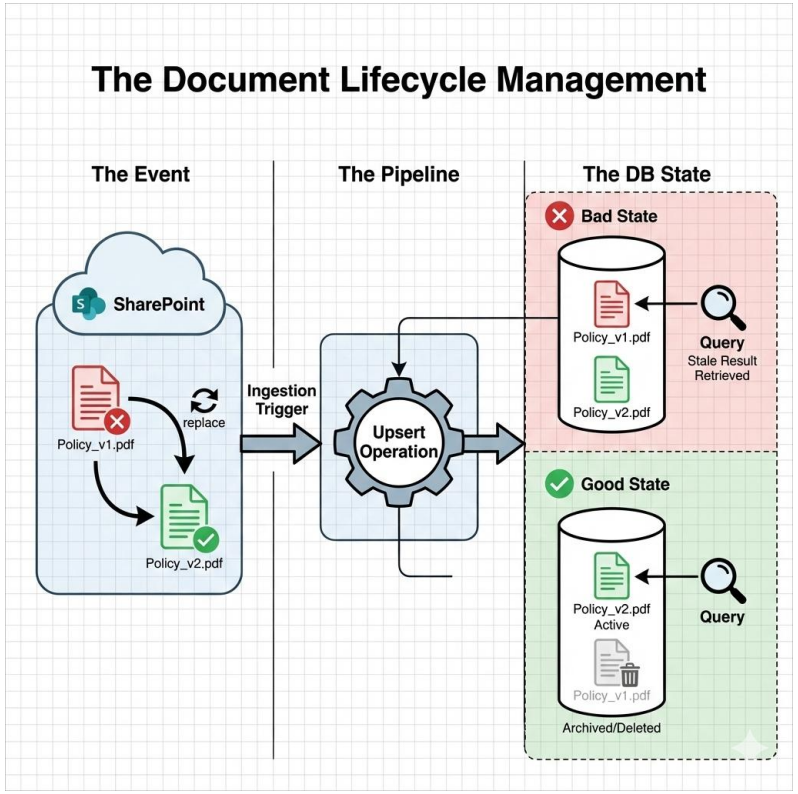
No access control

Vector DBs are not permission engines. They return anything semantically relevant.

© 2025 TekFrameworks

Handling "Stale" Data

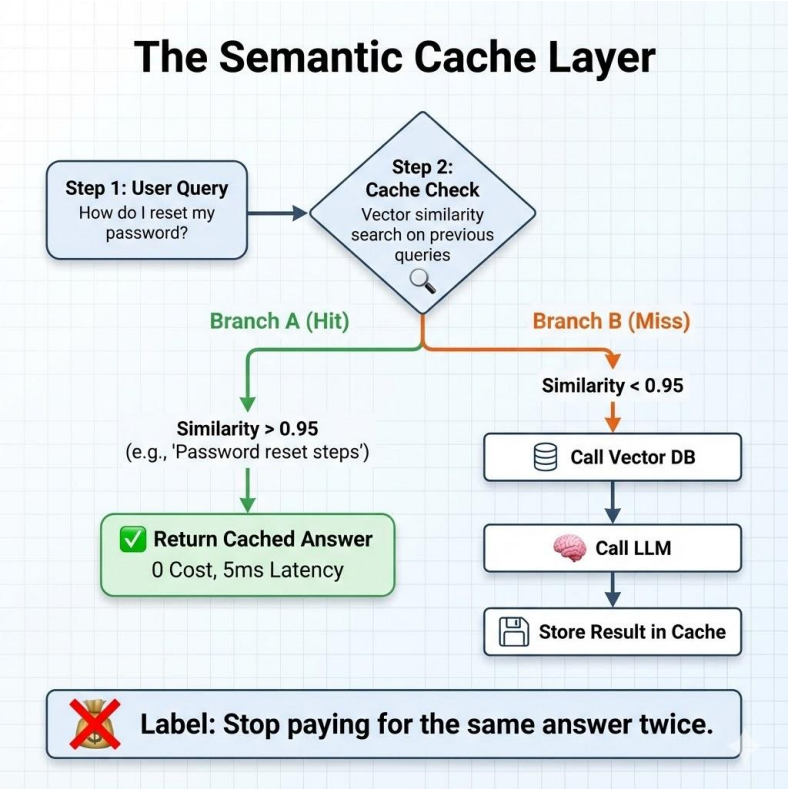
- **The “Tuesday Problem”**
  - HR updates “Remote work policy” on Monday. On Tuesday, a user asks about it. The RAG system retrieves the old policy because its still in index
- **Solution 1: Upsert Strategy**
  - Delete-then-insert: When a document is updated, you must identify all old chunks (via parent_doc_id) and delete them before adding new ones. Don’t just append!
- **Solution 2: Recency Weighting (Soft sorting)**
  - Don't just ingest text. Ingest Validity Dates
  - If you must keep old versions (for audit), use a **Decay Function**.
  - Boost the score of newer documents so they appear at the top of the context window



© 2025 TekFrameworks

Cost & Latency Strategy

- Problem:
  - LLM inference is expensive and slow
  - In Enterprise (eg HR bots), 30-40% of queries are repetitive (eg: Holiday list, Reset password)
- Solution: Semantic Caching (Redis/GPTCache)
  - Vector Similarity on the query itself (not string matching)
  - If *Distance(New Query, Cached Query) < Threshold*, serve the cached LLM response
  - *Result:* Reduces cost and latency to near-zero for common questions



© 2025 TekFrameworks

Hybrid Search — Keyword + Vector

Vector search finds meaning. Keyword search finds exact terms. Production systems need both.

BM25 — sparse retrieval

Finds exact keyword matches.

Reliable for part numbers, codes, proper nouns.

Vector — dense retrieval

Finds semantically related content.

Reliable for conceptual queries.

EnsembleRetriever

Weighted merge of both.

Tune weights based on your domain.

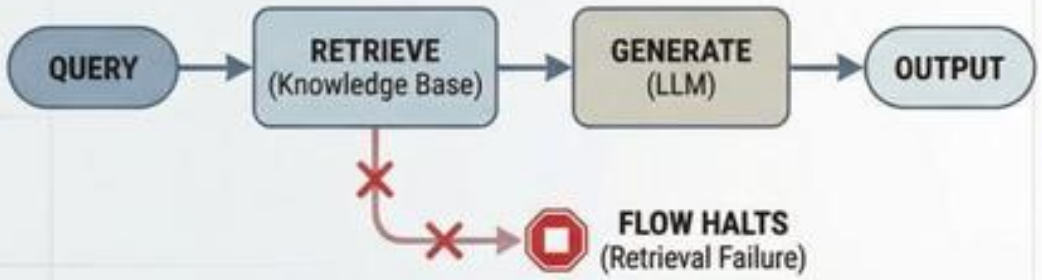
```
from langchain.retrievers import (
    BM25Retriever,
    EnsembleRetriever)
from langchain.vectorstores import FAISS

# Sparse — finds exact terms
bm25 = BM25Retriever.from_texts(
    doc_list)
bm25.k = 5  # top-5 keyword matches

# Dense — finds meaning
faiss = FAISS.from_texts(
    doc_list, embeddings)

# Blend both with weights
ensemble = EnsembleRetriever(
    retrievers=[bm25, faiss],
    weights=[0.4, 0.6]
)  # tune: more BM25 for ID-heavy domains
```

© 2025 TekFrameworks

CLASSIC RAG VS. AGENTIC GRAPHS

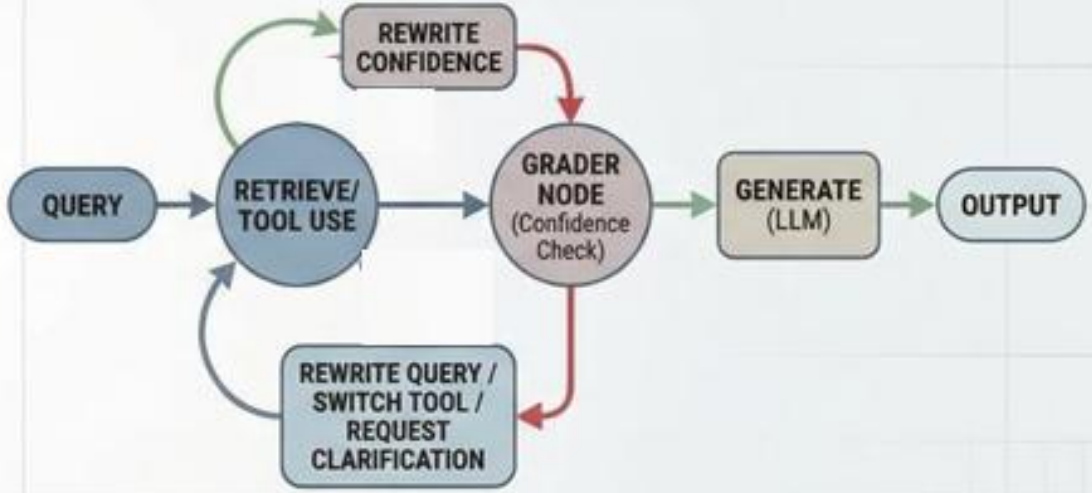
Evolution from Linear Chains to Cyclic Confidence-Based Systems

CLASSIC RAG: LINEAR CHAIN



- Linear retrieve-then-generate pipeline.
- Single-pass process.
- System fails if initial retrieval is insufficient.
- No mechanism for error correction or refinement.

AGENTIC GRAPHS: CYCLIC & ADAPTIVE



- Adds cyclic confidence checks and decision points.
- Iterative process: Rewrites queries, switches tools, or prompts user.
- Continues until grader node is satisfied with evidence.
- Robust, self-correcting, and dynamic.

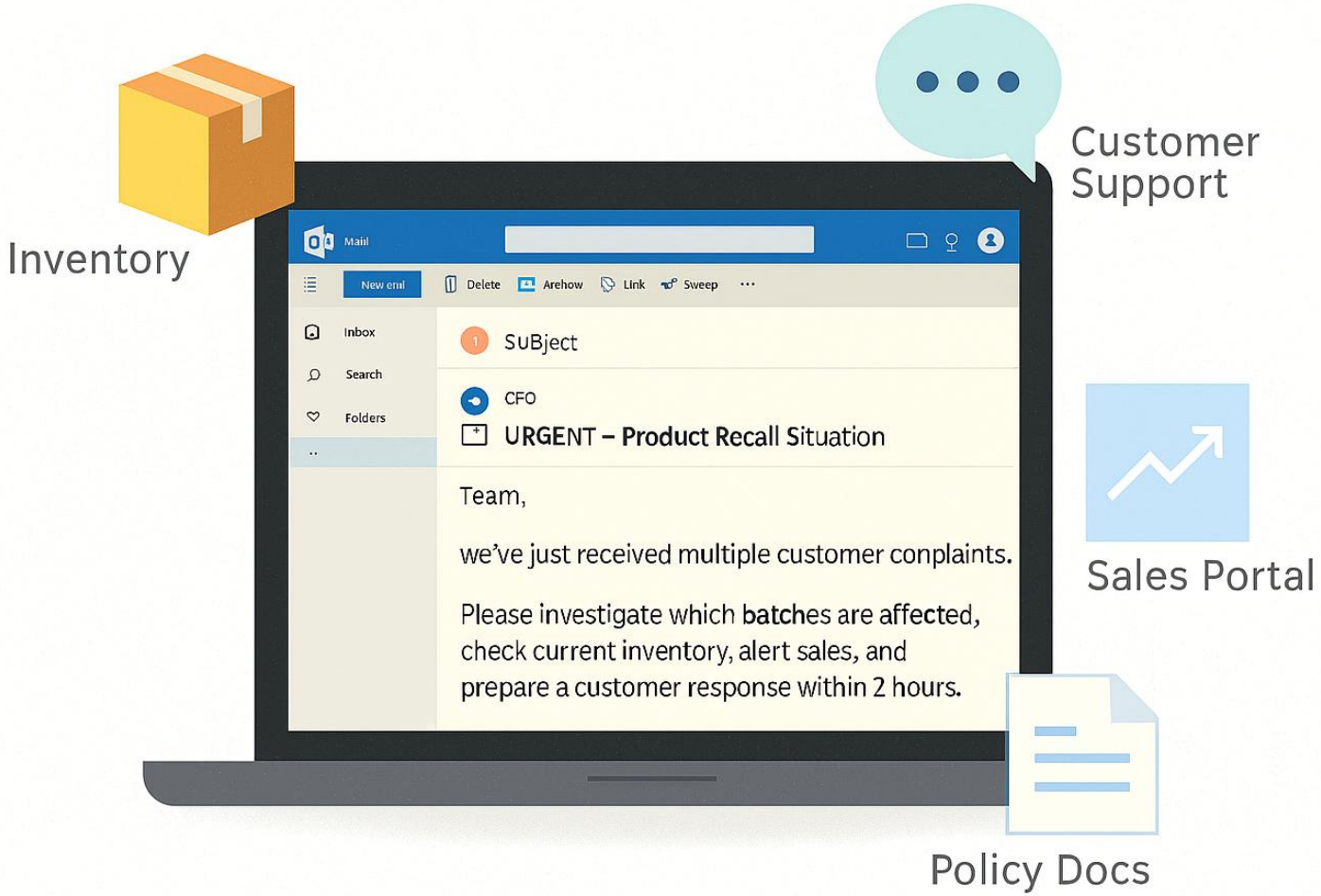
Strategic shift from brittle, one-shot systems to resilient, self-improving architectures.

© 2025 TekFrameworks

Agents: Intelligent Systems

© 2025 TekFrameworks

Workflow automation isn’t enough…

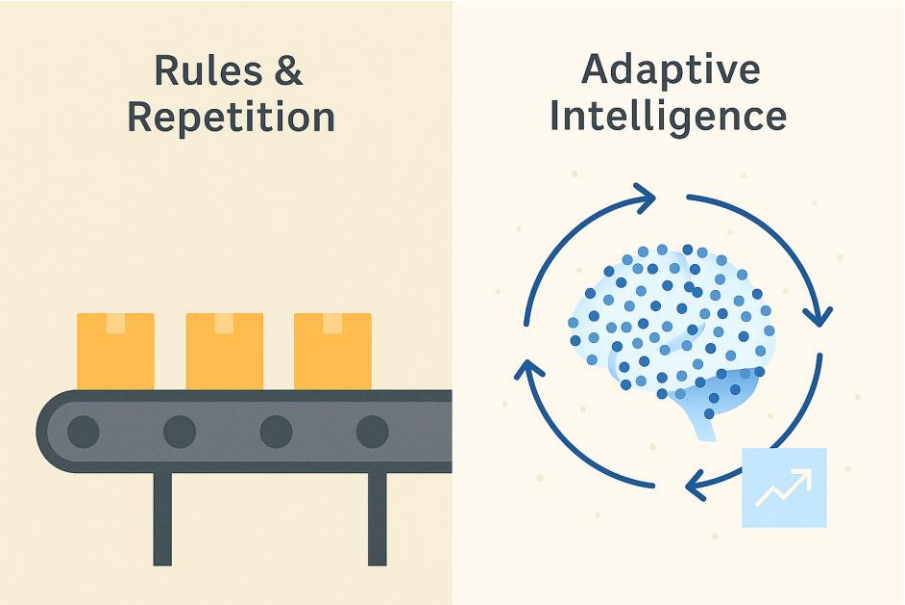


© 2025 TekFrameworks

Workflow vs Agentic flow

- Workflows follow instructions
- Workflows are predictable
- A → B→ C

- Agents are flexible
- Agents reason and adapt
- Dynamic Plan→Act→Reflect loop

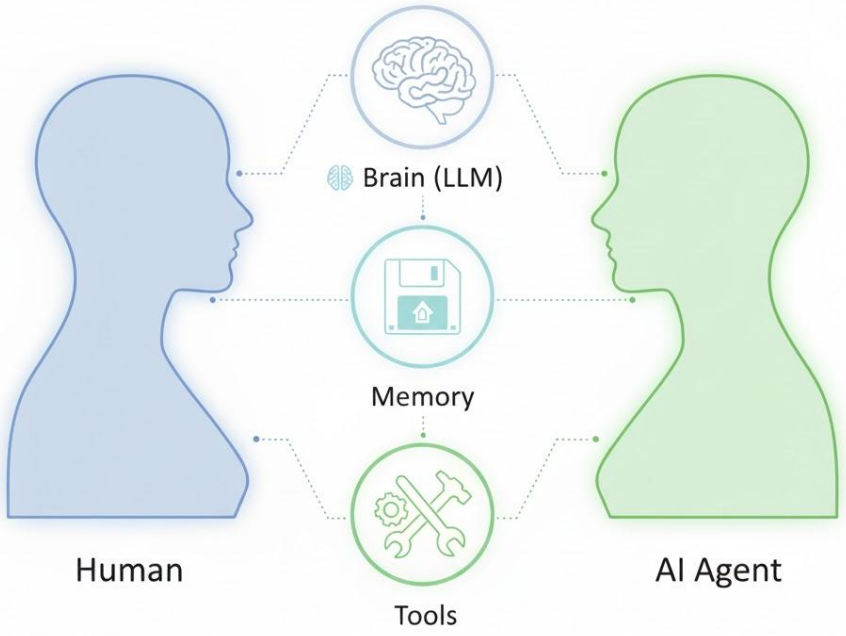


Workflow automation checks the weather and emails it every morning.
An agent can respond to “Should I bring an umbrella today?”. Understands the *why* you asked and decides to check the weather.

© 2025 TekFrameworks

Agent: Digital employee

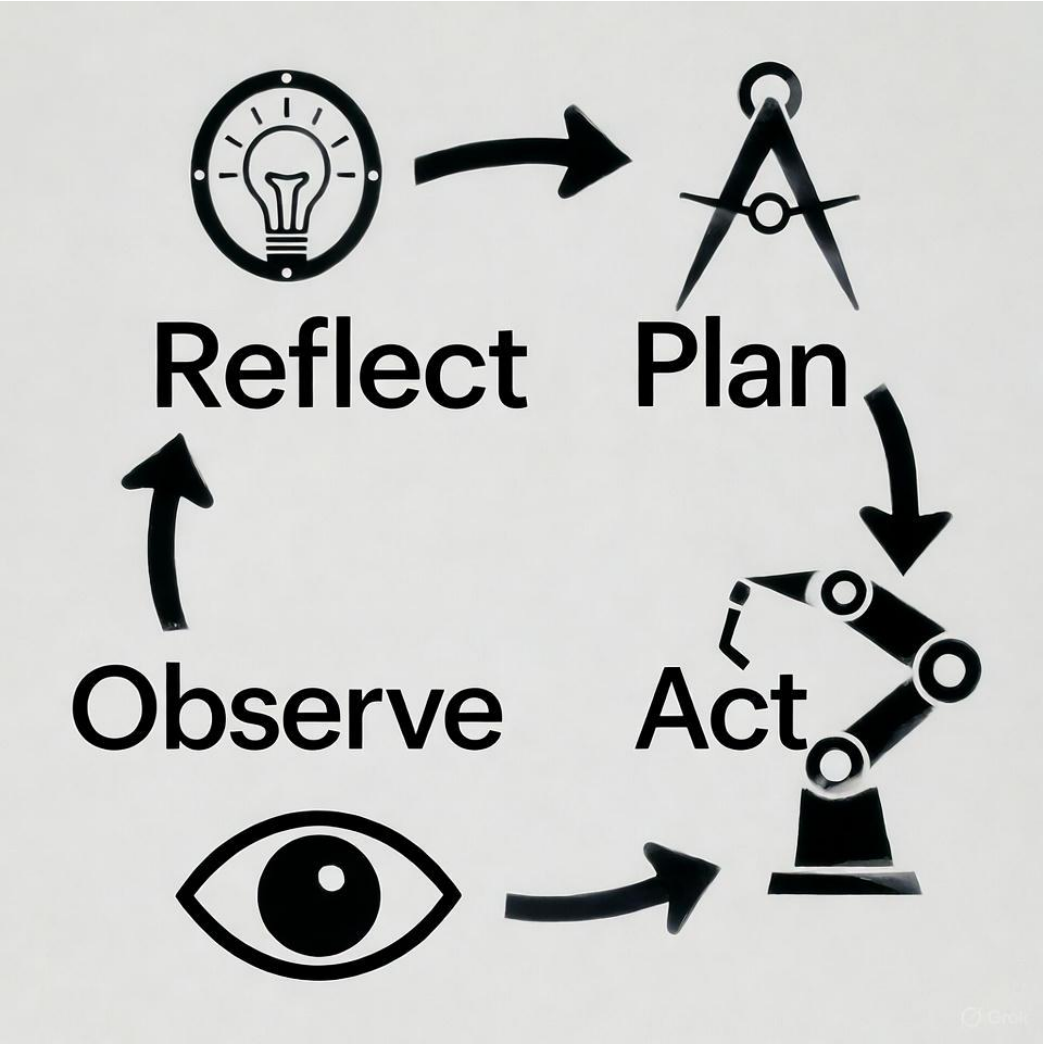
- **Brain:** Reasoning & language understanding
- **Memory:** Context & history
- **Tools:** APIs, apps, data sources



© 2025 TekFrameworks

How intelligent systems reason

- **Plan:** Form strategy using context & memory
- **Act:** Use tools / APIs to execute
- **Observe:** Collect results & feedback
- **Reflect:** Assess outcome & re-plan if needed



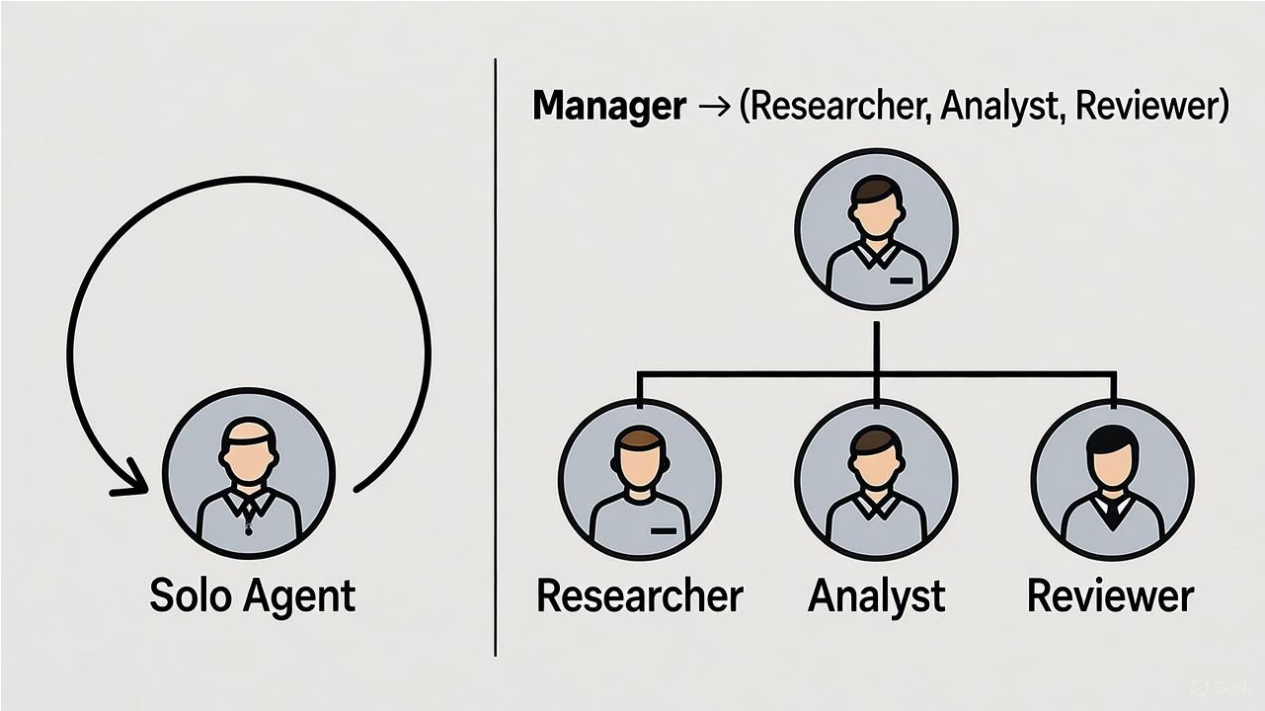
© 2025 TekFrameworks

Anatomy of an agentic flow

- Brain (LLM): Reasoning & language understanding
- Memory: Remembers context & state
- Tools: APIs / systems for action
- Controller: Coordinates the loop
- Reflection: Evaluates & improves results
- Guardrails: Ensure safety & compliance

© 2025 TekFrameworks

Single vs Multi-Agent Systems

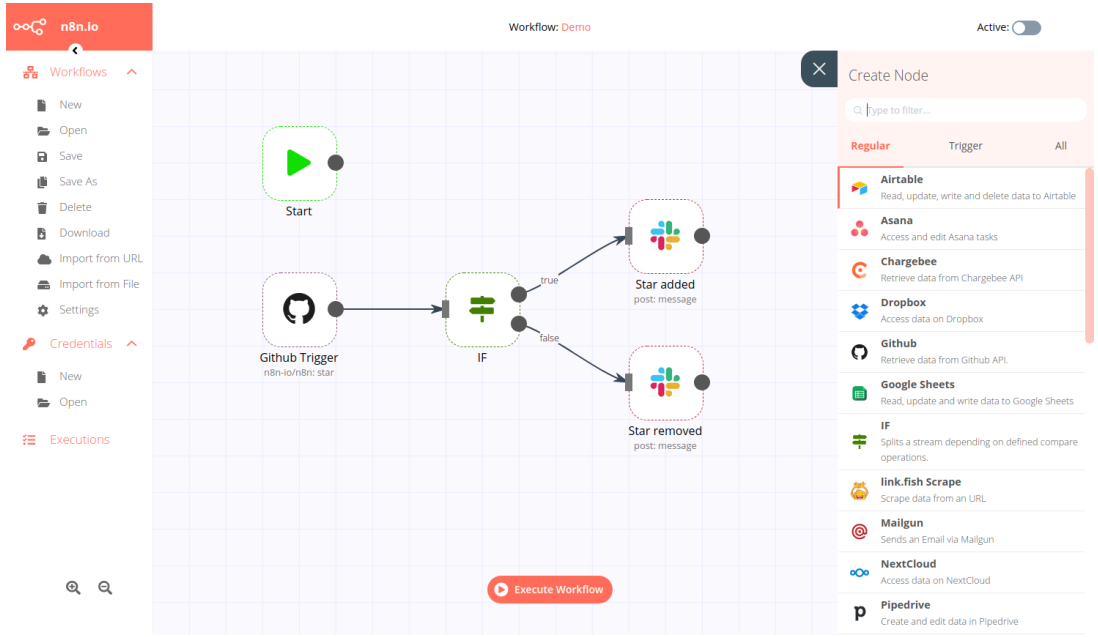


- **Single Agent:** Best for focused, bounded tasks.
- **Multi-Agent:** When tasks need specialization & coordination
- **Supervisor Agent:** Plans and delegates; others execute and report back

© 2025 TekFrameworks

n8n: A Visual Agent Builder

- Drag-and-drop nodes for prompts, memory, tools, retrievers
- Each node = a step in the agentic loop
- Connect, run, and observe results in real time



© 2025 TekFrameworks

Strands: AWS's Open-Source Agent SDK

Every concept we just covered maps directly to a Strands construct.

- **Tool** @tool decorator — any Python function becomes a callable tool
- **Agent loop** Agent() — instantiate with a model, tools, and system prompt
- **MCP connector** MCPClient() — discovers remote tools from an MCP server at runtime
- **Guardrails** add_hook() — intercepts invocations before the LLM is called
- **Observability** result.metrics — token usage, cycle time, tool call counts

© 2025 TekFrameworks

The @tool Decorator

Any Python function becomes an agent tool with one decorator. The docstring is what the agent reads to decide when to use it.

```
from strands import tool

@tool
def my_tool(query: str) -> str:
    """
    Describe WHAT this tool does and WHEN to use it.
    The agent reads THIS docstring — not the function name —
    to decide whether to call this tool.
    A vague docstring = wrong tool selection.
    """
    # Your logic here — call an API, query a DB,
    # run a calculation, anything Python can do.
    result = do_something(query)
    return str(result)  # always return a string
```

© 2025 TekFrameworks

RAG is one tool in the agent's toolbox.

Now let us build the toolbox.

An agent wraps an LLM with tools it can call, memory it can read, and a loop that keeps running until the task is done. RAG becomes one tool among many — the knowledge retrieval tool.

Knowledge
(RAG tool)

Action
(API / DB tool)

Reasoning
(LLM loop)

© 2025 TekFrameworks

Wrapping RAG as a Strands Tool

The full RAG pipeline from Day 1 becomes an agent tool in a few lines. The agent now has access to the HR knowledge base.

```
from strands import tool

@tool
def hr_rag_search(query: str) -> str:
    """
    Search the enterprise HR knowledge base.
    Use for: leave policy, remote work rules,
    contractor policy, attendance, HR documents.
    Always call this for HR knowledge questions.
    """
    # rag_pipeline() = the full Day 1 pipeline:
    # embed → retrieve → rerank → generate
    result = rag_pipeline(query)

    # rag_pipeline returns (answer, chunks)
    # agent only needs the answer string
    return str(result[0]) if isinstance(
        result, tuple) else str(result)
```

© 2025 TekFrameworks

Creating a Strands Agent

An agent is an LLM with a system prompt and a list of tools. Strands runs the ReAct loop automatically.

```
from strands import Agent

agent = Agent(
    # Nova Lite — lightweight, strong instruction-
    # following, good reasoning. Used throughout lab.
    model="amazon.nova-lite-v1:0",

    tools=[hr_rag_search],  # tools the agent can call

    system_prompt="""
    You are an enterprise HR assistant.
    For any HR policy or document question,
    always call hr_rag_search.
    Never answer from your own knowledge.
    """
)

# One call — Strands runs the full ReAct loop
response = agent("What is remote work eligibility?")
print(response.message)
```

© 2025 TekFrameworks

Adding an Action Tool

A second tool gives the agent the ability to act, not just answer. Routing between tools is automatic — driven by intent.

```
@tool
def apply_leave(days: int,
                reason: str = 'Not specified',
                employee_id: str = 'EMP001') -> str:
    """
    Apply leave for the current employee.
    Use when the user wants to REQUEST or APPLY leave.
    days = number of leave days requested.
    """
    leave_requests.append({
        "employee_id": employee_id,
        "days": days, "reason": reason
    })
    return f'Leave of {days} day(s) applied.'

# Agent routes by intent — no if/else needed
agent = Agent(model='amazon.nova-lite-v1:0',
    tools=[hr_rag_search, apply_leave], ...)
```

© 2025 TekFrameworks

Multi-Step Orchestration

One user query can trigger multiple tools in sequence. The agent plans and executes the steps automatically.

```
# A single compound request
response = agent(
    "Check the leave policy and apply
     leave for 2 days due to personal work."
)

# What Strands runs inside the ReAct loop:
# Cycle 1 — agent calls hr_rag_search()
#           retrieves leave policy text
#           appends result to context

# Cycle 2 — agent calls apply_leave(
#               days=2, reason='personal work')
#           stores the request
#           appends confirmation to context

# Final  — agent synthesises both results
# into one coherent answer for the user
print(response.message)
```

© 2025 TekFrameworks

Memory and State Management

Agents need memory to be useful across a conversation. Strands gives you two layers.

Conversation memory

Short-term — within this session

The message history passed to the LLM on every call. This is how the agent knows what was said earlier. Strands manages this automatically within a session.

Application state

Long-term — persists beyond the LLM call

Data the tools read and write — in the lab, leave_requests is the in-memory state. In-memory resets on restart. For production: persist to a DB or file.

© 2025 TekFrameworks

Context Engineering — What Is Actually in Your Context Window

TekFrameworks

Prompt engineering is what you say. Context engineering is what the model sees. For production agents, the second one determines whether it works.

The six things consuming your context window

1 System prompt

Role, rules, guardrails. Static but expensive.

2 Tool descriptions

Every @tool docstring the agent can see. Static but grows with each tool you add.

3 User message

The query. Small — but triggers everything else.

4 Retrieved resources

RAG chunks injected per query. Highly variable in size.

5 Assistant messages

Every prior response re-sent each turn — LLMs are stateless.

6 Tool call history

Every tool input and output from prior cycles in this run.

The bloat problem

- System prompt + tools
- Retrieved chunks (RAG)
- Conversation history
- Tool call history
- ⚠ Remaining space

Filling 100% of the context window degrades performance. Optimal is 60–70% capacity. More context is not always better.

© 2025 TekFrameworks

Local tools work.

But in enterprise, tools need to be services.

If you build an apply_leave tool today and another team builds a payroll agent next month — should they reimplement the tool from scratch, or just call yours?

MCP makes tools into discoverable, shareable services.

© 2025 TekFrameworks

The integration problem

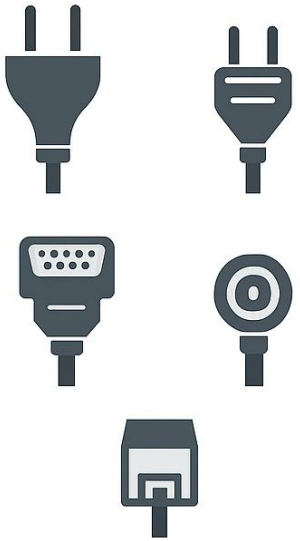
- Agents are powerful only if they can act
- But enterprise systems are closed, authenticated, and siloed
- Need a *secure, standardized way* for LLMs to access tools & data

© 2025 TekFrameworks

From APIs to MCP

- APIs expose functions; MCP standardizes how agents discover & call them
- Introduced by Anthropic, now an open ecosystem standard
- MCP defines tool capabilities, authentication, and data format

API =
Plug Type Chaos
(Different shapes, data formats)



MCP =
USB-C for AI Agents
(Universal connector for tools)

© 2025 TekFrameworks

The Old World — Tools Baked In

```
# main.py  — tools hardcoded & imported

search_tool    = get_tavily_tool()
financial_tool = get_financial_data_tool()
news_tool      = get_yahoo_finance_news_tool()

researcher = get_researcher(
    search_tool,
    image_downloader_tool,
    llm=default_llm
)
finance_agent = get_finance_agent(
    financial_tool,
    news_tool,
    llm=default_llm
)
```

🔒 **Locked in**

Tools exist only inside this project. No other agent can find or reuse them.

🔁 **Copy-paste trap**

Every new agent reimplements the same integrations from scratch.

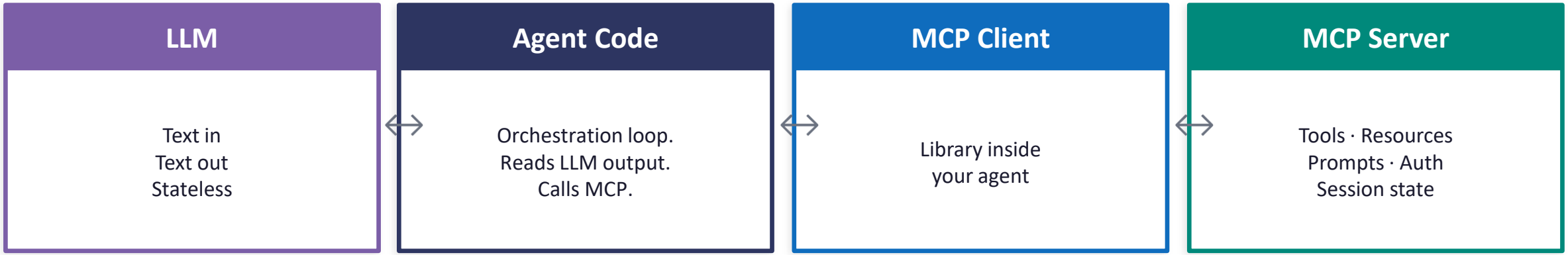
🔌 **Plug chaos**

Each tool has its own auth, format, error handling. Custom glue code everywhere.

👻 **No discoverability**

Your agent must know exactly which tools exist at build time — no runtime flexibility.

What MCP Is — The Architecture



MCP never talks to the LLM directly

MCP talks to your agent code. Agent code talks to the LLM. The LLM is a stateless text processor — it only knows what you put in its context window. Agent code must fetch capabilities from MCP and inject them into the prompt before every LLM call.

MCP connection IS stateful

The MCP server holds session context between calls. Agent code ↔ MCP Server remember each other across the full workflow — fetch portfolio, check margin, place order — all one session.

© 2025 TekFrameworks

Meet the Example

"Should I buy more HDFC Bank today?"

To answer this, our Zerodha trading agent needs:

📂 RESOURCES (data to read)	🔧 TOOLS (actions to take)
R1 Live HDFC Bank price & order book	T1 `place_order(ticker, qty, price)`
R2 Your portfolio holdings & avg. buy price	T2 `set_stop_loss(order_id, trigger)`
R3 Latest analyst ratings & target prices	T3 `log_trade_rationale(text)`
R4 Your available cash / margin balance	T4 `send_notification(channel, msg)`

© 2025 TekFrameworks

Step 1 — Capability Discovery (1/2: What the Server Returns)

The agent's first move: ask the MCP server what it can do

```
{
  "tools": [
    { "name": "place_order",       "description": "Place a buy/sell order on NSE/BSE" },
    { "name": "set_stop_loss",     "description": "Set a stop-loss trigger on an order" },
    { "name": "send_notification", "description": "Send a trade alert to the user" }
  ],
  "resources": [
    { "uri": "zerodha://portfolio",  "description": "Current holdings and avg. buy prices" },
    { "uri": "zerodha://marketdata", "description": "Live prices, order book, depth" },
    { "uri": "zerodha://margin",     "description": "Available cash and margin balance" }
  ]
}
```

© 2025 TekFrameworks

Step 1 — Capability Discovery (2/2: What It Means for Your Agent)

The JSON goes to your agent code — not to the LLM. Here's what that means:

🔍 Runtime discovery — not build-time hardcoding

The agent asks at runtime, not at deploy time. Add a new tool to the MCP server tonight — every agent that calls get_capabilities() tomorrow will know about it. No code changes, no redeployment.

⚠ The LLM does not receive this JSON directly

The JSON lands in your agent code. Agent code must reformat it into readable text and inject it into the system prompt. The LLM only ever sees text — it has no idea MCP exists. (We will see this action in code shortly)

🔁 Swap the server — agent adapts automatically

Zerodha replaced by Angel One? Point your MCP client at the new server URL. Next capabilities call returns the new tool list. The agent reconstructs its prompt automatically. Your orchestration code stays unchanged.

© 2025 TekFrameworks

Step 2 — Resource Fetch

The LLM decides what data it needs — the agent goes and gets it

LLM reasoning:

"User asks about buying HDFC Bank. I can see resources: portfolio, marketdata, margin. I need all three before I can give a recommendation. Fetch them."

R1: Portfolio

`GET zerodha://portfolio`

HDFCBANK: 150 shares @ ₹1,580 avg
Total invested: ₹2,37,000

R2: Market Data

`GET zerodha://marketdata`

HDFCBANK CMP: ₹1,642 (+1.8%)
52W High: ₹1,880 | P/E: 19.2

R3: Margin

`GET zerodha://margin`

Available cash: ₹85,400
Max buyable @ CMP: ~52 shares

© 2025 TekFrameworks

Step 3 — Tool Invocation

The LLM recommends an action — the agent decides whether to execute

```
# LLM returns a structured tool call recommendation:
{
  "tool": "place_order",
  "parameters": {
    "ticker":     "HDFCBANK",
    "exchange":   "NSE",
    "qty":        25,
    "order_type": "LIMIT",
    "price":      1638.00,
    "rationale":  "Stock is 12.6% below 52W high. Avg cost
                   ₹1,580. Adding at ₹1,638 improves DCA."
  }
}
```

Agent executes:

1. Validate parameters
2. Check margin (again)
3. POST to Zerodha API
4. Receive order_id
5. Call set_stop_loss()
6. Call send_notification()

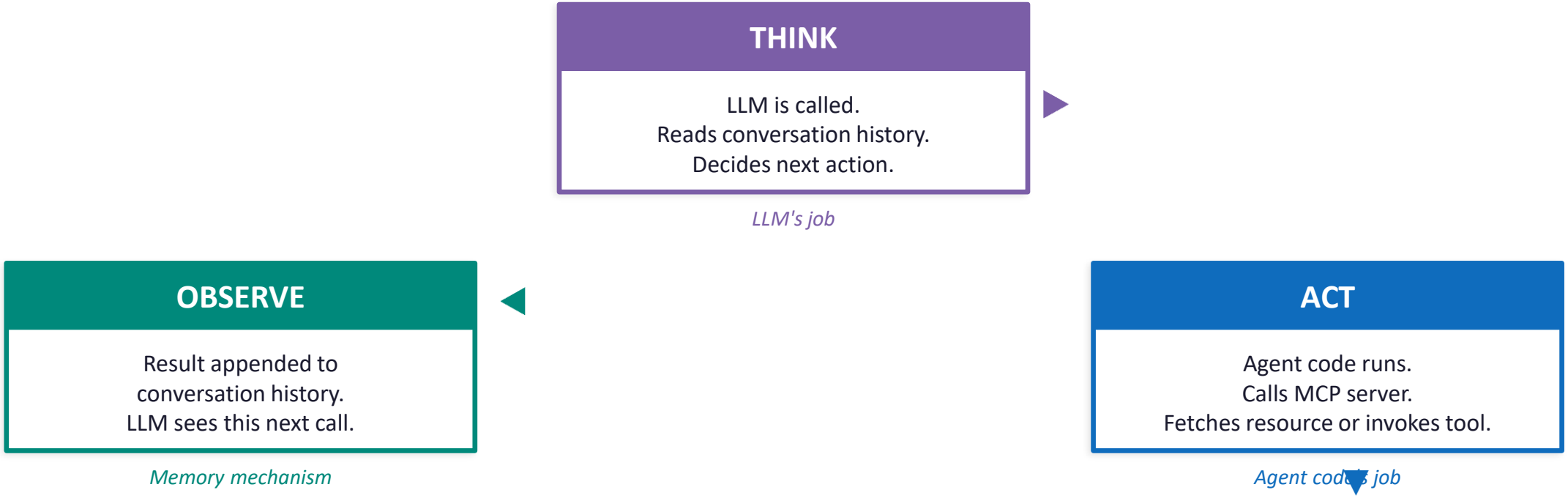
🛡 **The LLM never executes directly.**

Claude, GPT, Gemini — none will reach into your network and call an API. They return a recommendation. Your agent code decides whether to fire. This is where you add human approval gates, spend limits, compliance checks — before anything irreversible happens.

© 2025 TekFrameworks

The ReAct Loop — How the Agent Actually Thinks

ReAct = Reasoning + Acting. Every agentic framework runs this pattern.



EXIT: When THINK produces a final answer instead of an action → return to user

Zerodha example trace:

Think → fetch portfolio | Think → fetch market data | Think → fetch margin | Think → recommend order | Think → done

© 2025 TekFrameworks

How Many LLM Calls? The Honest Accounting

Every THINK step in the ReAct loop = one LLM call. Here's what that means for our example:

Call 1 — **Planning**
User question + capabilities list → LLM decides: fetch portfolio, market data, margin

Call 2 — **After portfolio fetch**
History + portfolio result → LLM decides: still need market data

Call 3 — **After market data fetch**
History + market data → LLM decides: still need margin

Call 4 — **After margin fetch**
History + all data → LLM decides: enough, here is my recommendation

Call 5* — **Tool confirmation (optional)**
If agent asks LLM to confirm before executing — one more call before place_order fires

Design principle: *For trading: always pre-fetch portfolio + price + margin in parallel (1 planning call). Dynamic only for the action phase.*

Static for data you always need. Dynamic only where the decision genuinely depends on what you find.

© 2025 TekFrameworks

MCP in Code — Server and Client

Server side (what Zerodha publishes):

```
from mcp.server import MCPServer
from mcp.types import Tool, Resource

server = MCPServer("zerodha-trading")

@server.tool()
def place_order(ticker: str, qty: int,
                price: float) -> dict:
    """Place a buy/sell order on NSE/BSE"""
    return zerodha_api.place_order(
        ticker, qty, price
    )

@server.resource("zerodha://portfolio")
def get_portfolio() -> str:
    """Current holdings & avg buy prices"""
    return zerodha_api.get_holdings()

server.run(host="0.0.0.0", port=8080)
```

Client side (your agent):

```
from mcp.client import MCPClient

# Connect once — session is stateful
client = MCPClient(
    "https://zerodha-mcp.broker.com"
)

# Discover capabilities at runtime
caps = client.get_capabilities()
# → formatted & injected into prompt
# → LLM reads formatted text

# Fetch a resource (triggered by LLM)
portfolio = client.get_resource(
    "zerodha://portfolio"
)

# Invoke a tool (triggered by LLM)
result = client.call_tool("place_order", {
    "ticker": "HDFCBANK",
    "qty": 25,
    "price": 1638.00
})
```

© 2025 TekFrameworks

The Zerodha MCP Server — Now in FastMCP

Same tools, same logic. FastMCP replaces the boilerplate with a decorator. The server becomes two lines of setup.

What changed from previous slide

`MCPServer()`	→ `FastMCP()`
`@server.tool()`	→ `@mcp.tool()`
`@server.resource()`	→ `@mcp.tool()` with fetch logic
`server.run(...)`	→ `mcp.run(transport='streamable-http')`

Everything else — tool logic, persistence, auth — stays identical.

```python
from mcp.server.fastmcp import FastMCP
import zerodha_api  # your existing client

# Same name your agent already knows
mcp = FastMCP("zerodha-trading")

# ── TOOL: place_order ─────────────────
@mcp.tool(description=
    "Place a buy/sell order on NSE/BSE")
def place_order(ticker: str,
                qty: int,
                price: float) -> dict:
    return zerodha_api.place_order(
        ticker, qty, price)

# ── TOOL: get_portfolio ───────────────
@mcp.tool(description=
    "Current holdings & avg buy prices")
def get_portfolio() -> str:
    return zerodha_api.get_holdings()

# ── START SERVER ──────────────────────
if __name__ == '__main__':
    mcp.run(transport='streamable-http')
    # → http://0.0.0.0:8000/mcp/
```

© 2025 TekFrameworks

Connecting Strands to the Zerodha MCP Server

The agent does not know or care how the server is built. It discovers tools at runtime, routes by docstring, executes by intent.

```
from strands.tools.mcp import MCPClient
from mcp.client.streamable_http import (
    streamablehttp_client)

# Point at the running FastMCP server
ZERODHA_MCP = 'http://zerodha-mcp.broker.com/mcp/'

# Lambda = lazy connection. Opens only
# when first tool call is made.
mcp_client = MCPClient(
    lambda: streamablehttp_client(ZERODHA_MCP)
)

# MCPClient fetches tool list at runtime:
#  place_order, get_portfolio, set_stop_loss,
#  send_notification — all discovered,
#  none hardcoded in the agent.
```

```
from strands import Agent

trading_agent = Agent(
    model='amazon.nova-lite-v1:0',

    # All Zerodha tools via MCP —
    # no local functions needed.
    tools=[mcp_client],

    system_prompt='''
    You are a Zerodha trading assistant.
    Use get_portfolio to read holdings.
    Check portfolio and margin before
    placing any order.
    Always set a stop-loss after ordering.
    '''
)
```

```
response = trading_agent("Should I buy more HDFC Bank today?")
# get_portfolio() → get_market_data() → check_margin() → place_order() → set_stop_loss() →
send_notification()
```

© 2025 TekFrameworks

But Now the Ask Gets Bigger...

Before: "Should I buy more HDFC Bank today?"

→ *One agent. MCP tools. Solved.* ✓

Now: "Rebalance my entire portfolio for a market downturn scenario."

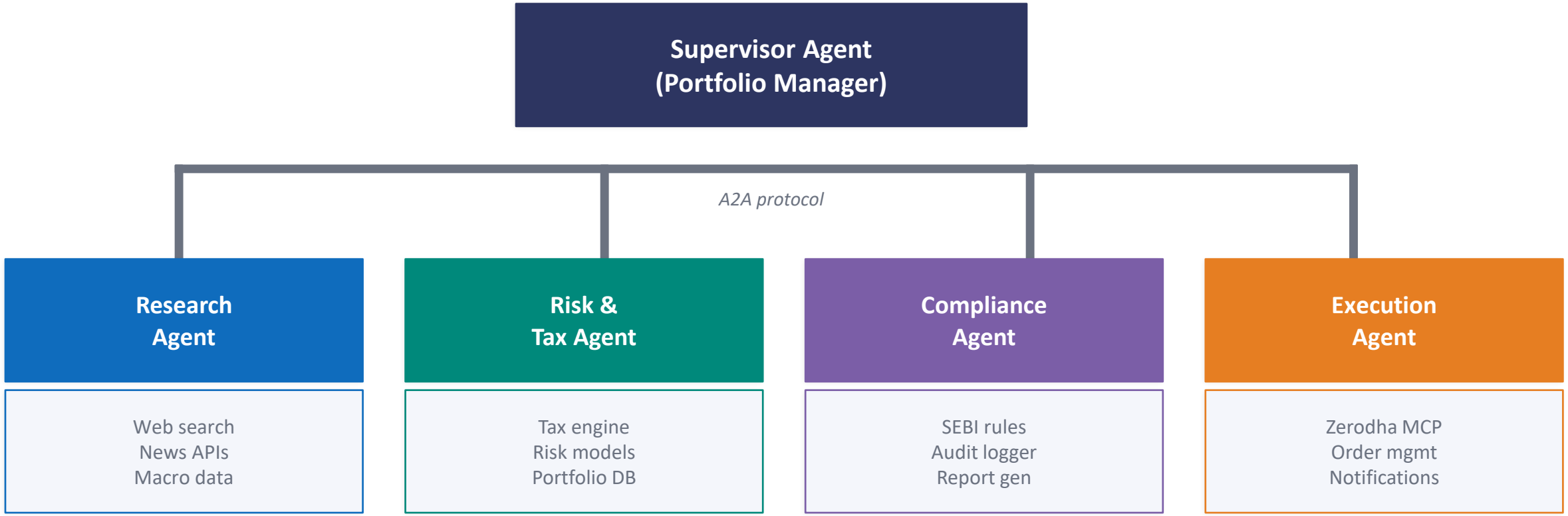
One agent is now being asked to:

- 📊 Analyse macro conditions & sector exposure
- 💼 Review every holding for risk/return
- 🧾 Calculate tax implications of each sale
- ⚖️ Verify SEBI regulatory limits
- 📈 Execute 12 trades in the right sequence
- 📋 Generate a compliance report

© 2025 TekFrameworks

Agent-to-Agent (A2A) — Agents as Services

Developed by Google with ecosystem partners | Agents discover, delegate, and coordinate across boundaries



Each agent is an independent service with its own tools (via MCP), its own LLM, its own ReAct loop. A2A is how they coordinate.

© 2025 TekFrameworks

A2A in Action — Portfolio Rebalancing

1

Supervisor → Research Agent

"Analyse HDFC, INFY, RELIANCE exposure vs sector benchmarks. Downturn scenario."

✓ Overweight INFY by 8%. HDFC within range. Reduce RELIANCE by 5%.

2

Supervisor → Risk & Tax Agent

"Given sell RELIANCE 200 shares — calculate tax liability and net proceeds."

✓ STCG ₹18,400. Net proceeds after tax: ₹1,12,600.

3

Supervisor → Compliance Agent

"Review proposed trades: Sell RELIANCE 200, Sell INFY 50, Buy HDFCBANK 40."

✓ All within SEBI limits. Approved. Audit log entry created.

4

Supervisor → Execution Agent

"Execute approved trade list. Confirmation required before each order."

✓ All 3 orders placed. Stop-losses set. Notification sent.

© 2025 TekFrameworks

A2A in Code — Portfolio Rebalancing

Each agent advertises itself. The supervisor discovers, delegates, and aggregates. No agent knows the others' internals.

```
# ── SUPERVISOR AGENT ──────────────────
# Each specialist publishes an Agent Card:
# name, description, tools, endpoint.

research  = A2AClient('http://research-agent/a2a')
risk_tax  = A2AClient('http://risk-agent/a2a')
compliance= A2AClient('http://compliance-agent/a2a')
execution = A2AClient('http://execution-agent/a2a')

# Step 1 — delegate analysis
analysis = research.send_task({
    'task': 'Analyse HDFC, INFY, RELIANCE
             vs sector benchmarks.',
    'scenario': 'market_downturn'
})
# → Overweight INFY 8%. Reduce RELIANCE 5%.

# Step 2 — delegate tax calculation
tax = risk_tax.send_task({
    'task': 'Calculate tax on selling
             RELIANCE 200 shares.',
    'holdings': analysis['proposed_trades']
})
# → STCG ₹18,400. Net proceeds ₹1,12,600.
```

```
# Step 3 — compliance check
approval = compliance.send_task({
    'task': 'Review proposed trades.',
    'trades': tax['approved_trades']
})
# → All within SEBI limits. Approved.

# Step 4 — execute (only after approval)
if approval['status'] == 'approved':
    result = execution.send_task({
        'task': 'Execute trade list.
                 Confirm each order.',
        'trades': approval['trades']
    })
# → 3 orders placed. Stop-losses set.

# Each agent runs its own ReAct loop.
# Each calls its own MCP tools.
# Supervisor only sees inputs + outputs.
# No agent knows the others' internals.
```

© 2025 TekFrameworks

The Clean Mental Model

Two protocols. One clear distinction.

MCP — Model Context Protocol	
Connects	Agent ↕ Tools & Data
Direction	Vertical (agent reaches down)
Analogy	USB-C — universal connector
In our example	Trading agent ↔ Zerodha APIs
Developed by	Anthropic (open standard)
State	Stateful session per connection

A2A — Agent-to-Agent Protocol	
Connects	Agent ↔ Agent
Direction	Horizontal (peers talk)
Analogy	Phone network for agents
In our example	Supervisor ↔ Specialist agents
Developed by	Google + ecosystem
State	Shared task context

MCP gives your agent hands. A2A gives your agent colleagues.

© 2025 TekFrameworks

The Enterprise Vision — Digital Departments

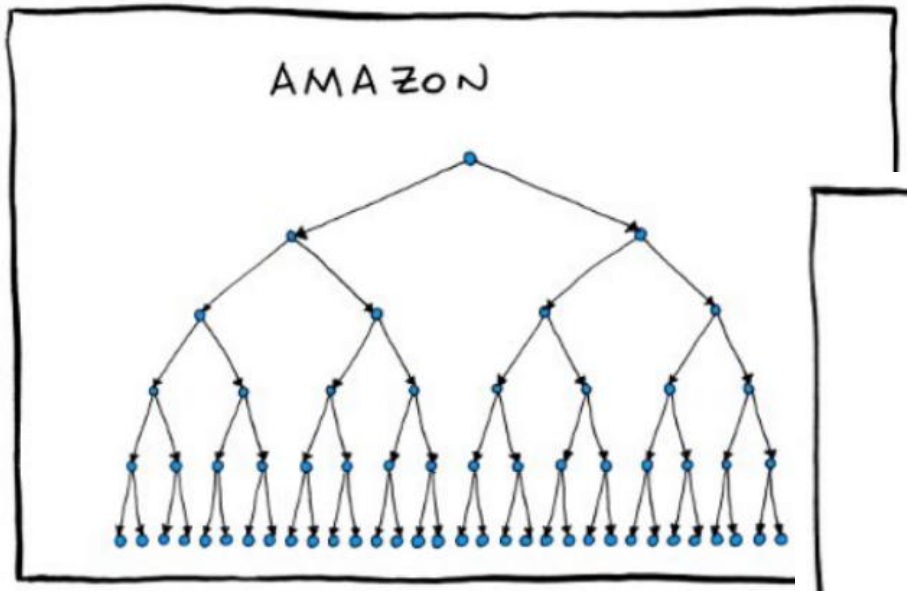
Trading Department	Risk & Compliance	Research Department	Client Relations
Portfolio Agent	Risk Agent	Analyst Agent	RM Agent
Execution Agent	Compliance Agent	Macro Agent	Report Agent
via MCP:	**via MCP:**	**via MCP:**	**via MCP:**
Zerodha MCP	Risk Model MCP	News MCP	CRM MCP
Market Data MCP	SEBI Rules MCP	Bloomberg MCP	Doc Gen MCP

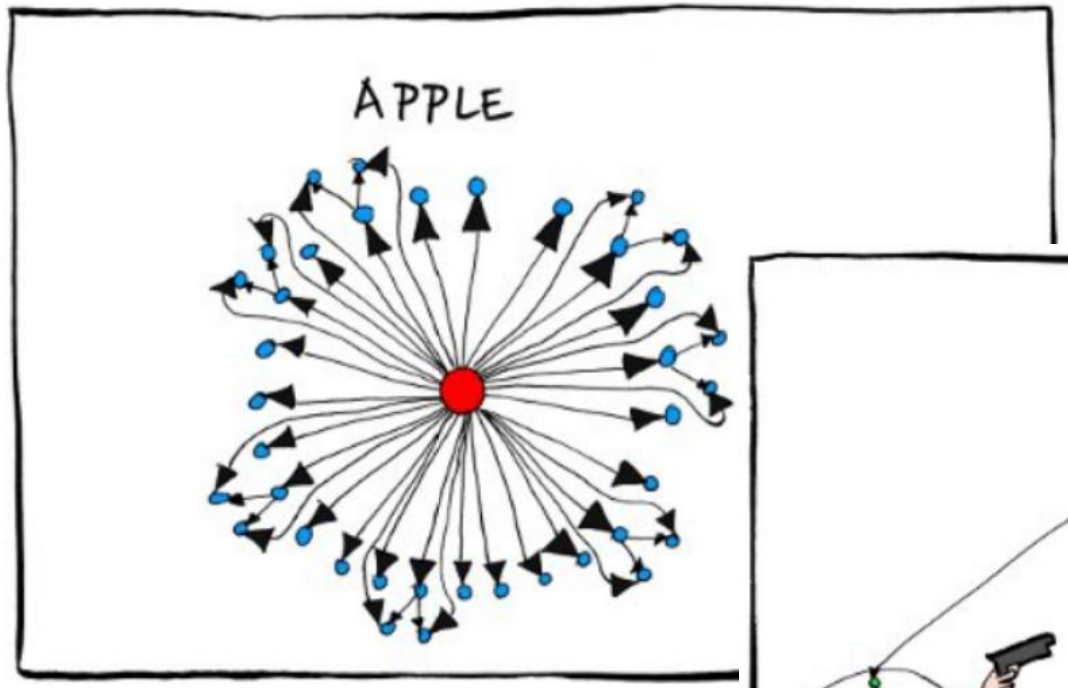
A2A Protocol Layer — departments coordinate just like real teams do

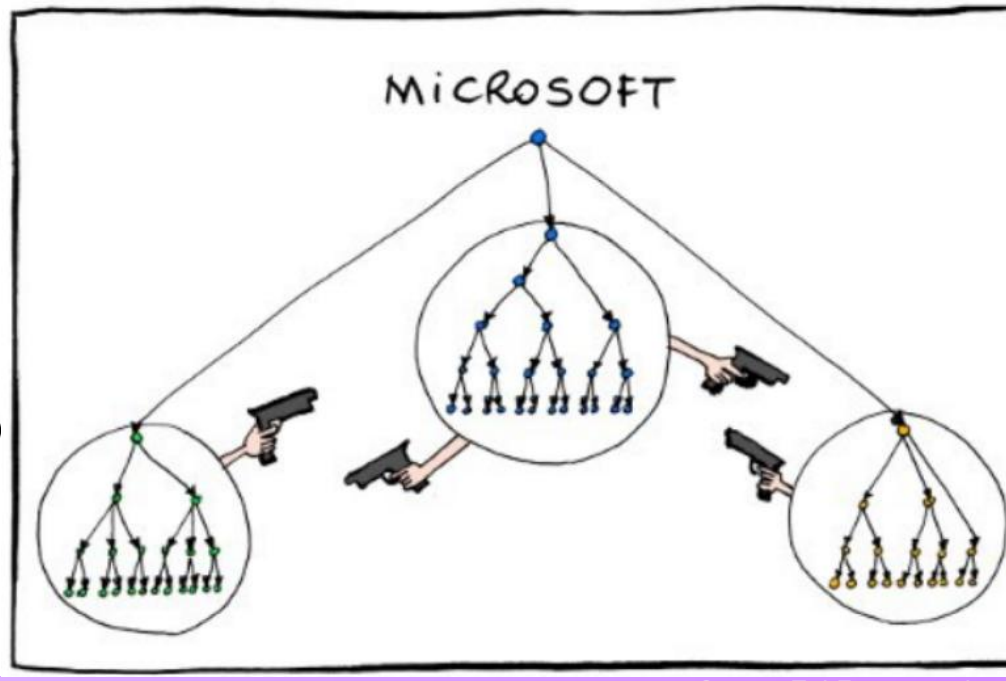
The same structure your firm has today — departments, specialists, protocols — **replicated in software.**

© 2025 TekFrameworks

Different Company Structures..



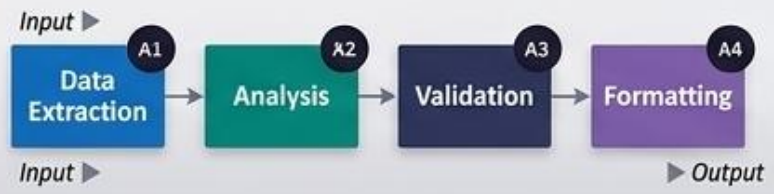




© 2025 TekFrameworks

COMPARATIVE OVERVIEW: AI AGENT ORCHESTRATION PATTERNS

Sequential Pattern



CORE PRINCIPLE

Linear, step-by-step pipeline. Output of one becomes input of the next.

WHEN TO USE

- Strict linear dependencies
- Step-by-step data transformations
- Progressive refinement

WHEN TO AVOID

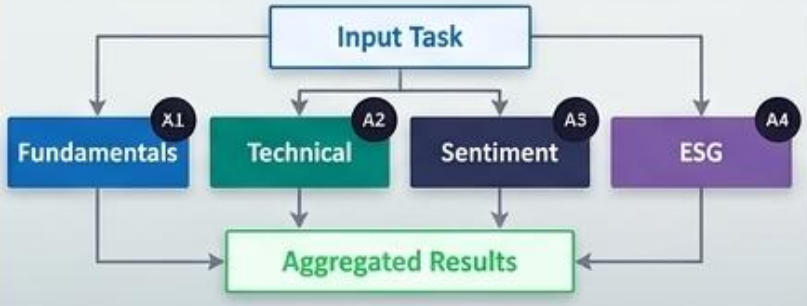
- Tasks can run in parallel
- High failure risk early
- Dynamic routing needed

ENGINEERING EXAMPLE

CI/CD Code Review Pipeline
(e.g., Lint –> Unit Test –> Security –> Build

PROGRESSIVE REFINEMENT

Parallel Pattern



CORE PRINCIPLE

Multiple specialist agents process the same input simultaneously. Results are aggregated.

WHEN TO USE

- Reduce wall-clock time
- Gain independent viewpoints
- Isolation of domains

WHEN TO AVOID

- Strong dependencies between steps
- Single agent can handle
- One-off task

ENGINEERING EXAMPLE

PR Review System
(e.g., parallel Security, Linting, Test Coverage, Docs checks)

UNMATCHED SPEED & DIVERSE PERSPECTIVE

Hierarchical Pattern

CORE PRINCIPLE

Top-down decomposition of complex tasks. Manager-worker delegation dynamic.

WHEN TO USE

- Large, complex tasks with multiple specialist domains
- Need centralized oversight
- Tasks have natural layered decomposition

WHEN TO AVOID

- Simple tasks
- Fast iteration is key
- When explicit authority is unnecessary

ENGINEERING EXAMPLE

Incident Response System
e.g., Supervisor delegates to Diagnostic, Remediation, and Notification agents

ORGANIZED COMPLEXITY & CENTRALIZED CONTROL

Managing Context at Scale

Decomposing agents is not only a capability decision. It is a context management strategy.

📦 Compaction

Before injecting a large retrieved document into the context window, use a fast cheap LLM call to summarise it first. A 50,000-token document becomes 500 words. The main agent sees the summary, not the raw text.

RAG retrieves a 40-page policy manual. Summarise to key clauses. Inject the summary.

🗄️ Local agent memory

Give each agent a small key-value store. Park large intermediate results — JSON payloads, API responses, fetched records — outside the active context window. Pull only what the next step actually needs.

Portfolio data fetched in step 1. Stored locally. Only the relevant ticker is passed to step 2.

🧩 Agent composition

When a task makes an agent's context balloon, split it. A sub-agent handles the heavy lifting — parsing documents, running calculations — and returns only a concise answer to the main agent. Each agent carries only its own context.

A2A: Research agent processes 30 tickers. Returns a ranked shortlist to the supervisor.

⚖️ Goldilocks system prompt

Too vague and the LLM wanders. Too prescriptive and you override its reasoning capability with rigid if/then logic. Define clear outcomes and broad approaches. Let the model reason about the steps.

Good: 'Always check margin before ordering.' Bad: 'If margin > X then do Y else do Z.'

© 2025 TekFrameworks

Agentic Frameworks — Choosing the Right Tool

TekFrameworks

All four are open-source and production-capable. Microsoft Agent Framework (Oct 2025) is the successor to both AutoGen and Semantic Kernel.

Feature	n8n	LangGraph	Microsoft Agent Framework	Strands
Maintained by	n8n GmbH	LangChain	**★ Microsoft**	**★ AWS**
Language	Low-code / JS	Python / JS	Python & .NET	Python
Visual Tooling	Built-in canvas	LangSmith	Foundry Studio	None — code-first
Cloud Support	Cloud / self-host	LangSmith	Azure AI Foundry	**★ AWS Bedrock**
Orchestration	Workflow-first	All types	All types	Tool-based ReAct loop
Enterprise Ready	Medium	High	**★ High (native)**	**★ High (native)**
Best For	Low-code automation	Stateful Python flows	Azure / MSFT stack	AWS-native agents + lab

★ Microsoft Agent Framework (public preview Oct 2025) is the official successor to both AutoGen and Semantic Kernel — both now in maintenance mode. ★ Strands released by AWS May 2025 — open-source.

● High / Available ● Partial / Medium

© 2025 TekFrameworks

Governance, Observability & Risk

© 2025 TekFrameworks

- Loops without guardrails → runaway API calls.
- Lack of reflection led to repeated failed tasks.
- Missing MCP validation → unsafe actions.
- Observability missing — couldn't trace what went wrong

"Governance isn't optional — it's survival"

We Spent $47,000 Running AI Agents in Production. Here's What Nobody Tells You About A2A and MCP.

Teja Kusireddy Follow 8 min read · Oct 16, 2025

686 11

Multi-agent systems are the future. Agent-to-Agent (A2A) communication and Anthropic's Model Context Protocol (MCP) are revolutionary. But there's a $47,000 lesson nobody's talking about: *the infrastructure layer doesn't exist yet, and it's costing everyone a fortune.*

© 2025 TekFrameworks

Governance: Keeping Agents Within Bounds

An agent that can act is an agent that can act incorrectly. Every tool added is a potential failure mode.

Wrong tool selection Agent calls apply_leave when user only asked a policy question

Hallucinated parameters Agent invents an employee_id or inflates the days count

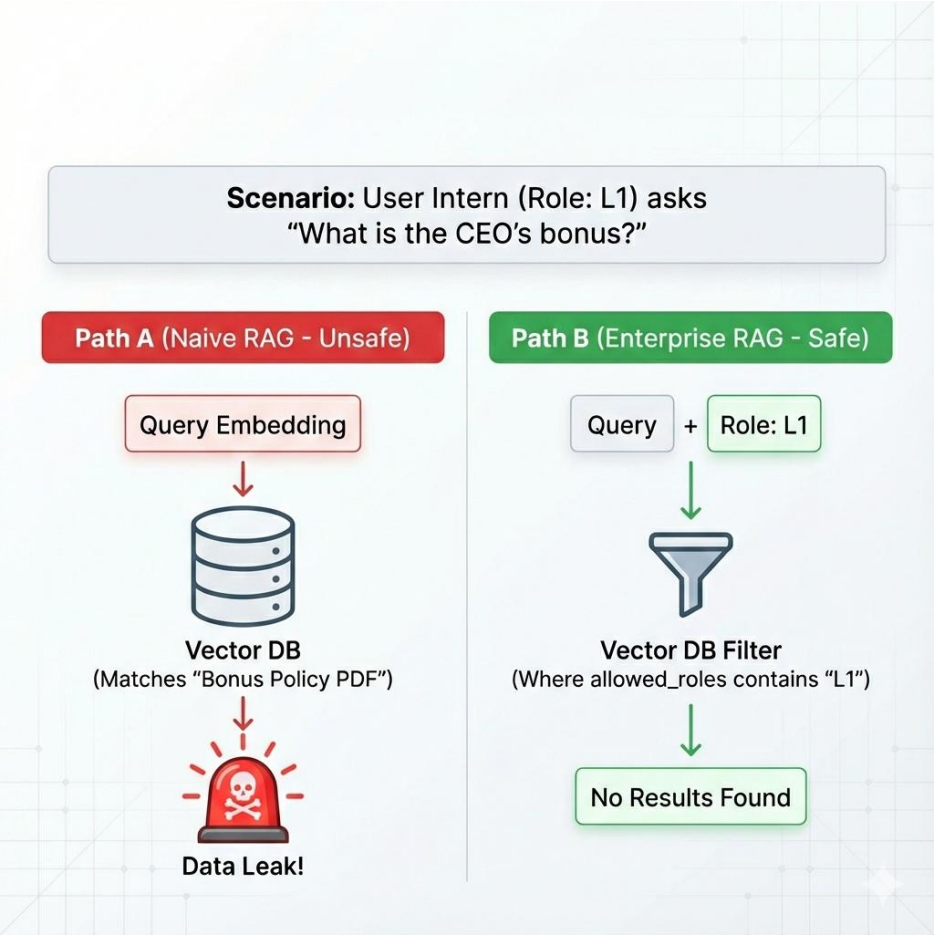
Unchecked actions Agent applies leave for a different employee without verifying

Sensitive data leakage Agent retrieves payroll data it should not have access to

© 2025 TekFrameworks

Security – Access control Lists

- Payroll Problem:
  - Vector DBs are not permission engines. They retrieve anything sematically relevant
  - Scenario: An intern asking "Executive compensation" might retrieve confidential salary data.
- Solution: Query-Time filtering (RBAC)
  - **Metadata is Mandatory:** Every document chunk must be tagged during ingestion with Access Control Lists (ACLs)
  - Example: *metadata: { "access_group": ["hr_admins", "execs"], "region": "US" }.*
  - **The Filter:** You **must** pass the user's identity (Role/JWT) into the retrieval query to apply a hard filter *before* the similarity search runs
- Case Study: Legal Tech (Harvey AI)
  - Law firms require "Ethical Walls"—Lawyer A cannot see Lawyer B's case files
  - Tenant-specific isolation or rigid metadata filters for every single query to ensure compliance



© 2025 TekFrameworks

Safety & Guardrails

- **Risk**:
  - A RAG system can be faithful, but still unsafe
  - Eg: Prompt injection, Historical bias in terminology
- **Metrics: Toxicity & Bias**:
  - **Toxicity:** Does the output contain hate speech, insults, or NSFW content? (Measured via BERT-based classifiers like Detoxify or OpenAI Moderation API)
  - **Bias/Fairness:** Does the model treat different groups differently? (e.g., Does it answer queries about "CEO" with "He" and "Assistant" with "She"?)
- Solution: Guardrails (NVIDIA NeMo/ Guardrails AI)
  - Don't rely on the LLM to be polite. Wrap it in a **Deterministic Firewall**
  - **Input Rail:** Scans user query for PII or malicious intent *before* it hits the LLM.
  - **Output Rail:** Scans the answer for specific banned words or patterns *before* showing it to the user.

© 2025 TekFrameworks

Strands Hooks — Deterministic Guardrails

Hooks intercept the agent before the LLM is called. This is a deterministic firewall — Python logic, not AI judgment.

```
from strands.hooks import BeforeInvocationEvent

BLOCKED = ['salary', 'payroll', 'ctc',
           'compensation', 'pay band']

def salary_guardrail(event: BeforeInvocationEvent):
    """Runs BEFORE every LLM invocation."""
    # Extract all text from incoming messages
    text = ' '.join(
        item['text']
        for msg in event.messages
        for item in (msg.get('content') or [])
        if isinstance(item, dict) and 'text' in item
    ).lower()

    if any(kw in text for kw in BLOCKED):
        raise ValueError(  # blocks the request
            'Policy block: salary info unavailable.')

# Register on the agent
agent.add_hook(salary_guardrail, BeforeInvocationEvent)
```

© 2025 TekFrameworks

Observability & Debugging — Seeing Inside Your Agent

TekFrameworks

Multi-agent systems fail in subtle ways. Observability is how you find out why.

LangSmith

Comprehensive tracing and evaluation for LLM apps. Full agent interaction logs, prompt diffs, token counts, and latency breakdowns per step.

LangFuse (open-source)

Self-hostable observability with cost tracking and automated evaluations. Python/JS SDK — drop-in if you can't send data to external SaaS.

Azure Application Insights

For Microsoft-stack deployments — instrument agent loops with OpenTelemetry; traces flow into App Insights dashboards natively.

Trace Multi-Agent Runs

Visualize full interaction graphs

Debug Transitions

Find where handoffs break down

Performance Analytics

Track latency & cost per step

Compare Strategies

A/B test orchestration patterns

Agent Orchestration Monitor

98.2%
Successful Handoffs

1.2s
Avg. Response Time

- **Orchestrator**
  → Planning task sequence
- **Data Agent**
  → Retrieved customer profile
- **Analytics**
  → Processed historical data
- **Content**
  → Generated response draft
- **QA Agent**
  → Validated compliance
- **Orchestrator**
  → Final approval pending

© 2025 TekFrameworks

Strands Metrics — Built-in Observability

Strands captures what happened inside every invocation. Metrics tell you what it cost, how long it took, and which tools were called.

```
# Run the agent — metrics captured automatically
result = agent('Apply leave for 2 days.')

# Token usage — total across all LLM calls
# (agents often call LLM 2-4x per user query)
print(result.metrics.accumulated_usage)
# {'inputTokens': 843, 'outputTokens': 67,
#  'totalTokens': 910}

# Cycle durations — one entry per ReAct loop cycle
print(result.metrics.cycle_durations)
# [1.23, 0.87]  ← two cycles: plan, then confirm
total = sum(result.metrics.cycle_durations)

# Tool metrics — calls and duration per tool
print(result.metrics.tool_metrics)
# {'apply_leave': {'calls': 1, 'duration': 0.12}}
```

© 2025 TekFrameworks

Day 2

From Knowledge Retrieval to Autonomous Action

Production RAG + Agentic Shift

- Stale data, cost, retrieval failures, access control — the production problems RAG alone does not solve.
- Hybrid search (BM25 + vector), semantic caching, ACL metadata filtering.
- Hard limit of RAG: some tasks require action, not just answers. This is where agents begin.
- Agent anatomy: brain, memory, tools, ReAct loop, guardrails.
- Workflows are linear and brittle. Agents are adaptive and self-correcting.

Building Agents — n8n + Strands

- n8n demo: ReAct loop made visible — plan, act, observe, reflect — without a line of code.
- @tool decorator: any Python function becomes an agent tool. The docstring is the routing contract.
- Strands Agent: model + tools + system prompt. The ReAct loop runs automatically.
- Multi-tool routing by intent. Multi-step orchestration in a single user query.
- Memory layers: conversation history (short-term) vs application state (long-term).

MCP, A2A, Governance

- MCP: tools become discoverable services. Capability discovery at runtime, not build time.
- FastMCP + Strands MCPClient: the Zerodha example end-to-end in production-ready code.
- A2A: agents coordinating with agents. Supervisor delegates. Each specialist owns its domain.
- Strands hooks: deterministic pre-invocation guardrails. Blocks before the LLM sees the query.
- Observability: token cost, cycle duration, tool metrics — per invocation, built into Strands.

© 2025 TekFrameworks

What ADP Can Build Now

The HR agent you built in the lab is not a demo. It is the template.

Knowledge tool

RAG over ADP's internal documents — policy manuals, compliance docs, runbooks, engineering wikis, support KB. Chunked, embedded, retrieved, reranked, grounded.

Action tools

Payroll systems, HR workflows, ticketing, incident management, deployment pipelines, onboarding. Each a @tool, callable by intent — no if/else routing logic.

MCP layer

One team builds the payroll tool. Every agent across ADP discovers and uses it. No reimplementation. No integration drift. Tools as shared infrastructure.

Governance layer

Hooks block sensitive queries before the LLM sees them. ACL filtering respects your existing permission model. Metrics give you cost and traceability per call.

The LLM is the reasoning engine. The pipeline around it is the product. You now know how to build both.

© 2025 TekFrameworks

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks